

Assessment Task: Profile Management System (Week 1 – Week 3)

1 Introduction

This project is all about a Profile Management System developed using IntelliJ, MySQLConnector, Java, MySQL Workbench. It allows users to retrieve and store profile data.

2 Objectives

The objectives are:

- To learn JDBC clearly
- To learn DAO patterns
- To connect Java with MySQL

3 Steps required

Step1: At first, I opened the MySQL Workbench.

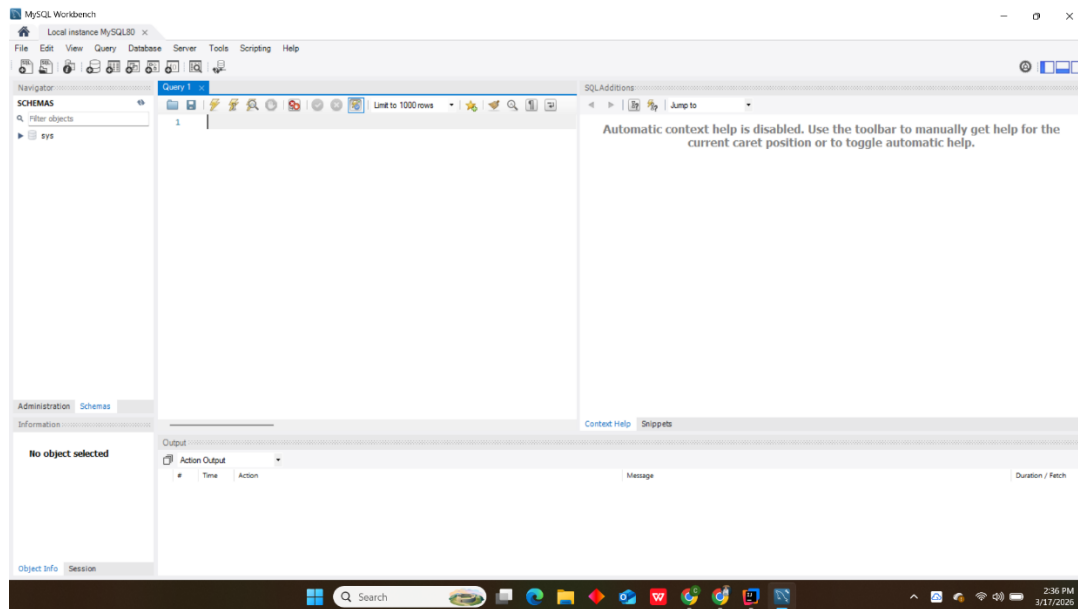


Figure 1 Opening the MySQL Workbench

Step 2: Creating a database user

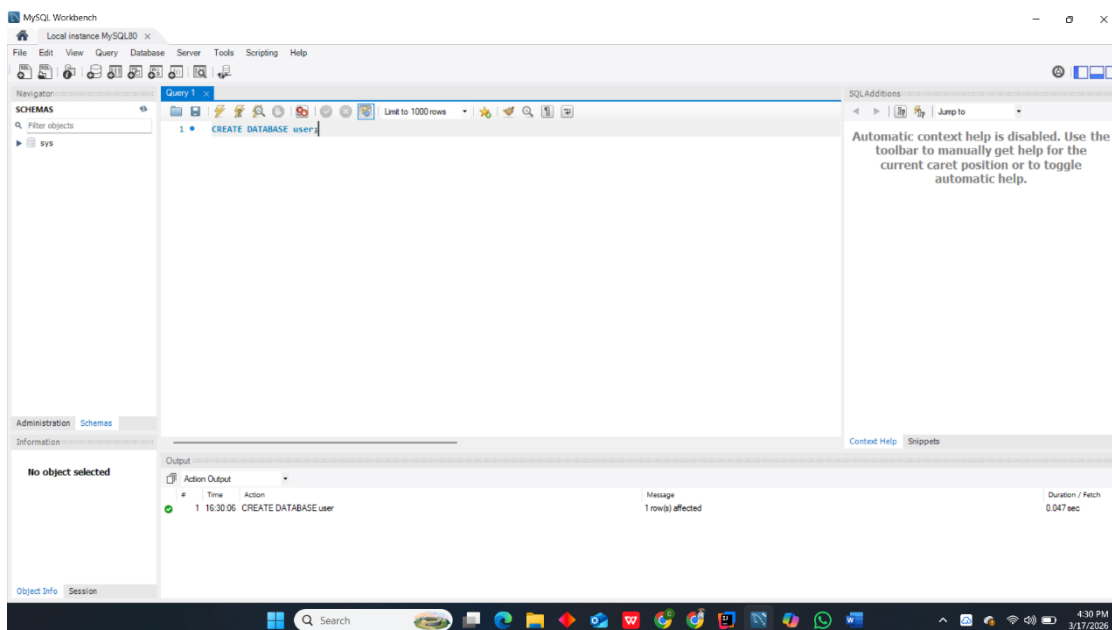


Figure 2 creating a database user

[Type here]

[Type here]

[Type here]

Step 3: Using the database user

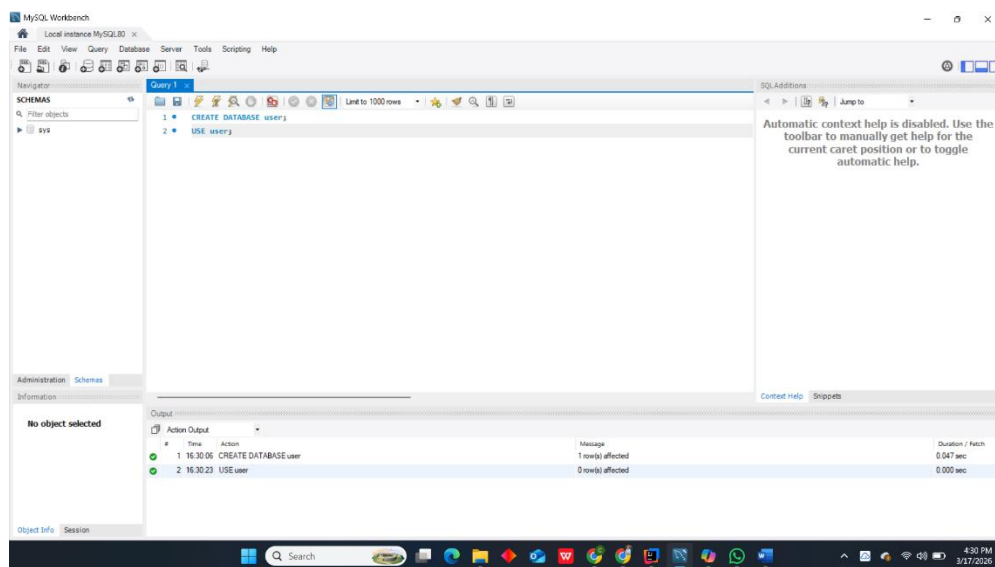


Figure 3 using the database user

Step 4: Creating a table named profile

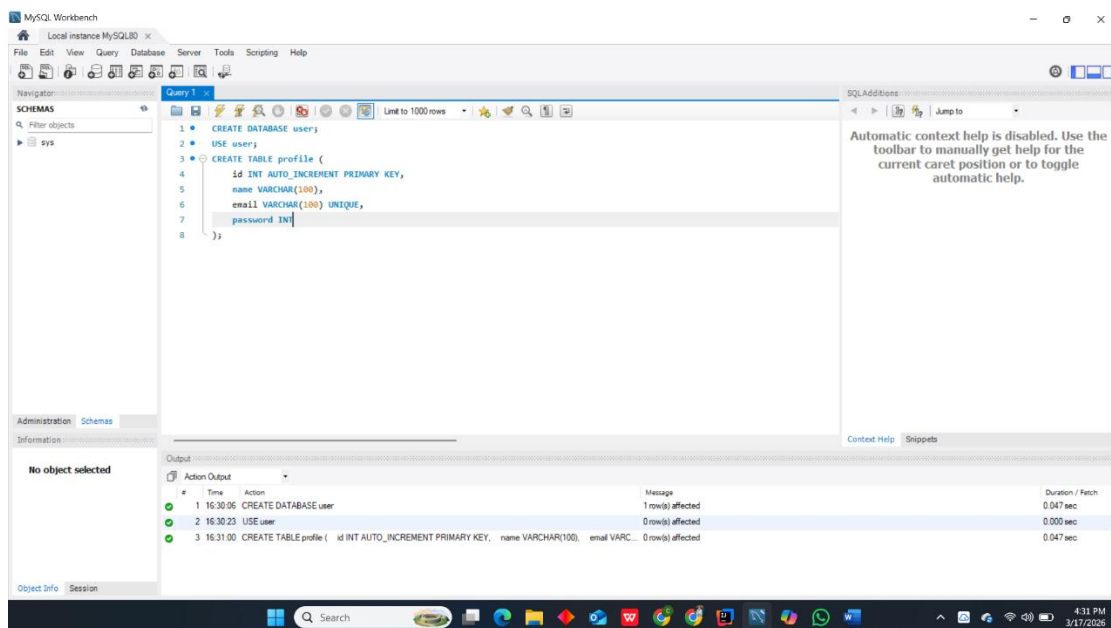


Figure 4 creating a table

[Type here]

[Type here]

[Type here]

Step 5: Showing if the table is created or not

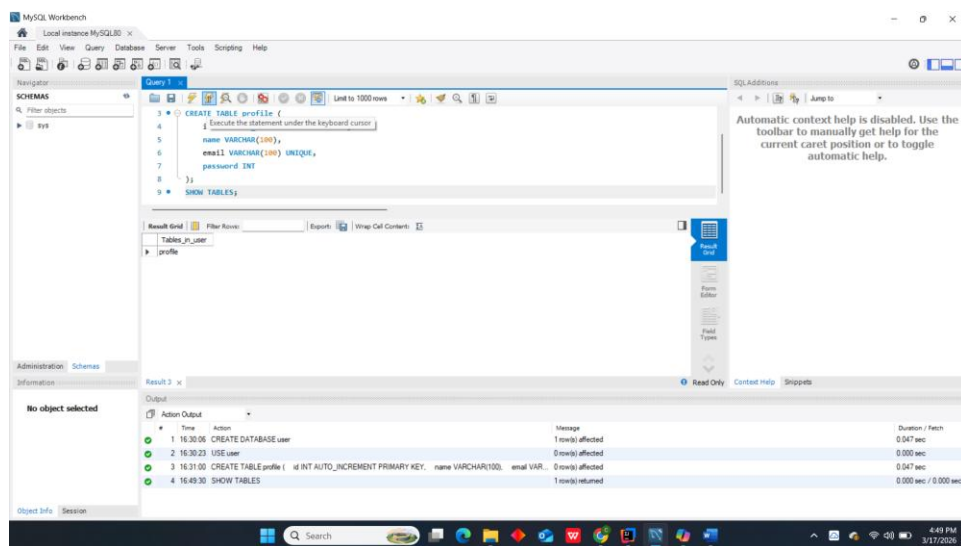


Figure 5 showing table

Step 6: Opening the IntelliJ and creating a java class packages

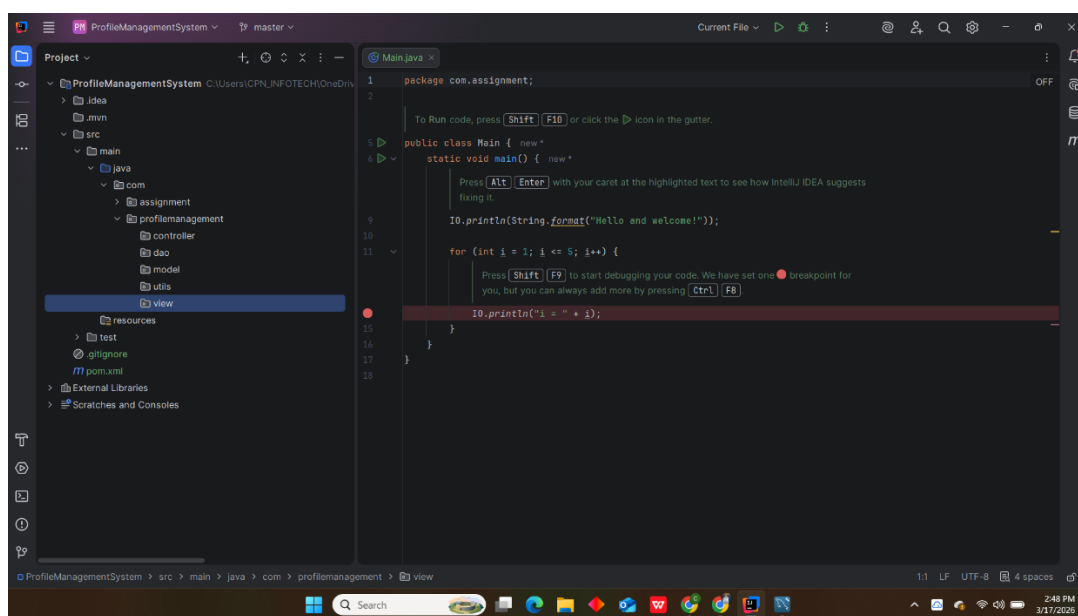


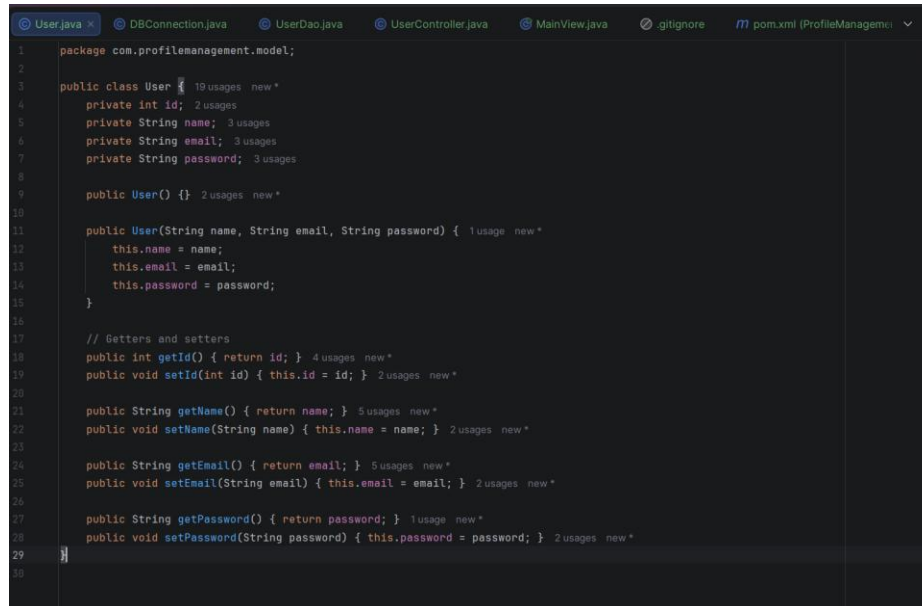
Figure 6 Creating the packages

[Type here]

[Type here]

[Type here]

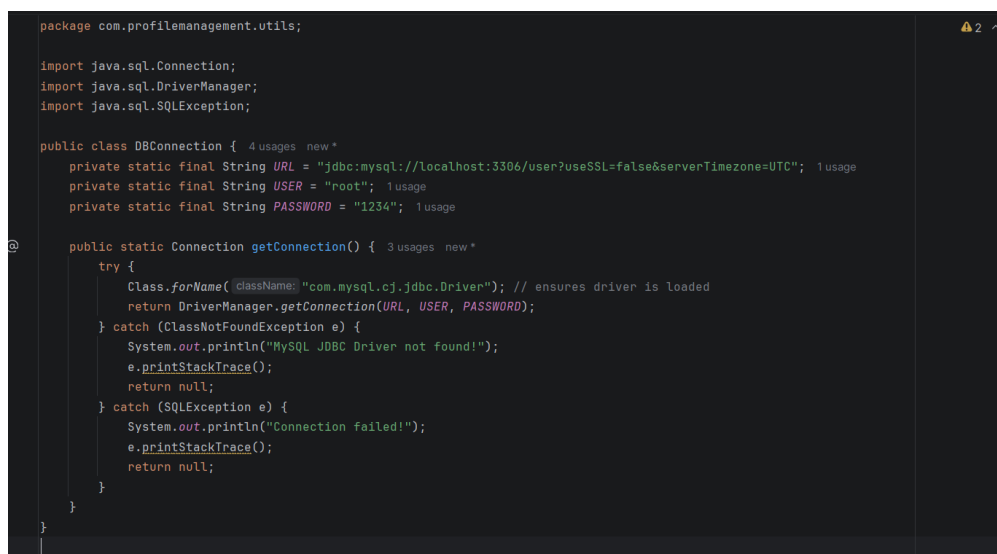
Step 7: User.java (Model)



```
1 package com.profilemanagement.model;
2
3 public class User {
4     private int id;
5     private String name;
6     private String email;
7     private String password;
8
9     public User() {}
10
11     public User(String name, String email, String password) {
12         this.name = name;
13         this.email = email;
14         this.password = password;
15     }
16
17     // Getters and setters
18     public int getId() { return id; }
19     public void setId(int id) { this.id = id; }
20
21     public String getName() { return name; }
22     public void setName(String name) { this.name = name; }
23
24     public String getEmail() { return email; }
25     public void setEmail(String email) { this.email = email; }
26
27     public String getPassword() { return password; }
28     public void setPassword(String password) { this.password = password; }
29 }
30
```

Figure 7 User.java(Model)

Step 8: DBConnecyion.java (Utils)



```
package com.profilemanagement.utils;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/user?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "1234";

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver"); // ensures driver is loaded
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException e) {
            System.out.println("MySQL JDBC Driver not found!");
            e.printStackTrace();
            return null;
        } catch (SQLException e) {
            System.out.println("Connection failed!");
            e.printStackTrace();
            return null;
        }
    }
}

```

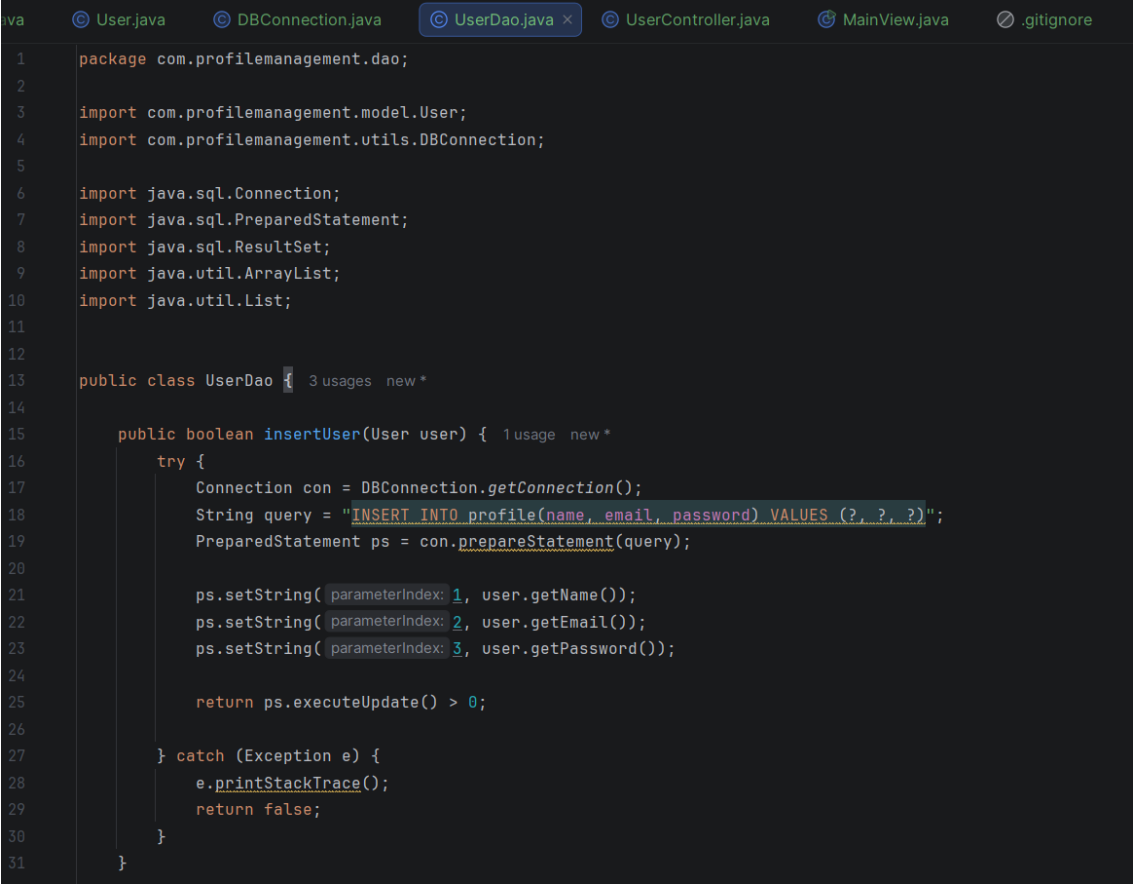
Figure 8 DBConnection.java (Utils)

[Type here]

[Type here]

[Type here]

Step 9: UserDao.java



```
1 package com.profilemanagement.dao;
2
3 import com.profilemanagement.model.User;
4 import com.profilemanagement.utils.DBConnection;
5
6 import java.sql.Connection;
7 import java.sql.PreparedStatement;
8 import java.sql.ResultSet;
9 import java.util.ArrayList;
10 import java.util.List;
11
12
13 public class UserDao { 3 usages new *
14
15     public boolean insertUser(User user) { 1 usage new *
16     try {
17         Connection con = DBConnection.getConnection();
18         String query = "INSERT INTO profile(name, email, password) VALUES (?, ?, ?)";
19         PreparedStatement ps = con.prepareStatement(query);
20
21         ps.setString(1, user.getName());
22         ps.setString(2, user.getEmail());
23         ps.setString(3, user.getPassword());
24
25         return ps.executeUpdate() > 0;
26     } catch (Exception e) {
27         e.printStackTrace();
28         return false;
29     }
30 }
31 }
```

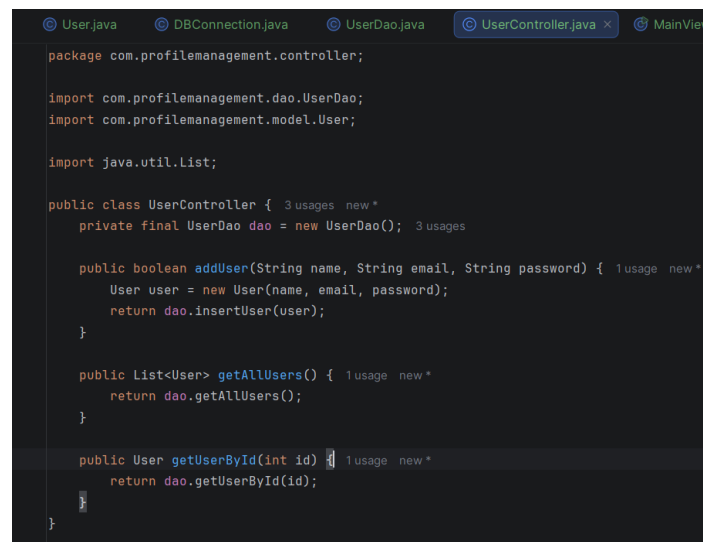
Figure 9 UserDao.java 1



```
13 public class UserDao { 3 usages new *
15     public boolean insertUser(User user) { 1 usage new *
30     }
31 }
32
33 public List<User> getAllUsers() { 1 usage new *
34     List<User> list = new ArrayList<>();
35     String sql = "SELECT * FROM profile";
36
37     try (Connection con = DBConnection.getConnection();
38         PreparedStatement ps = con.prepareStatement(sql);
39         ResultSet rs = ps.executeQuery()) {
40
41         while (rs.next()) {
42             User u = new User();
43             u.setId(rs.getInt( columnLabel: "id"));
44             u.setName(rs.getString( columnLabel: "name"));
45             u.setEmail(rs.getString( columnLabel: "email"));
46             u.setPassword(rs.getString( columnLabel: "password"));
47             list.add(u);
48         }
49     } catch (Exception e) {
50         e.printStackTrace();
51     }
52     return list;
53 }
54
55
56 > public User getUserById(int id) {...}
80 }
```

Figure 10 UserDao.java 2

Step 10: UserController.java



```

package com.profilemanagement.controller;

import com.profilemanagement.dao.UserDao;
import com.profilemanagement.model.User;

import java.util.List;

public class UserController {
    private final UserDao dao = new UserDao();

    public boolean addUser(String name, String email, String password) {
        User user = new User(name, email, password);
        return dao.insertUser(user);
    }

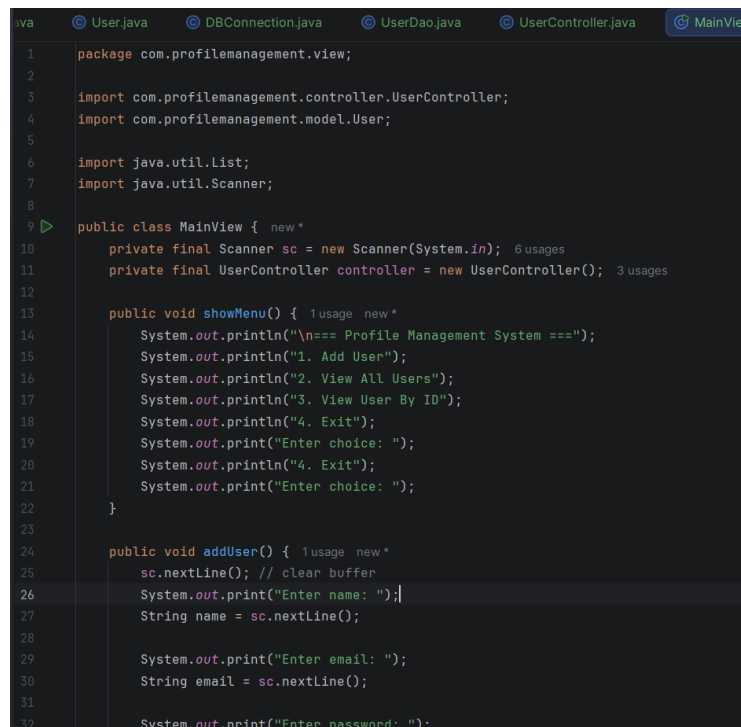
    public List<User> getAllUsers() {
        return dao.getAllUsers();
    }

    public User getUserById(int id) {
        return dao.getUserById(id);
    }
}

```

Figure 11 UserController.java

Step 11: MainView.java



```

package com.profilemanagement.view;

import com.profilemanagement.controller.UserController;
import com.profilemanagement.model.User;

import java.util.List;
import java.util.Scanner;

public class MainView {
    private final Scanner sc = new Scanner(System.in);
    private final UserController controller = new UserController();

    public void showMenu() {
        System.out.println("\n== Profile Management System ==");
        System.out.println("1. Add User");
        System.out.println("2. View All Users");
        System.out.println("3. View User By ID");
        System.out.println("4. Exit");
        System.out.print("Enter choice: ");
        System.out.println("4. Exit");
        System.out.print("Enter choice: ");
    }

    public void addUser() {
        sc.nextLine(); // clear buffer
        System.out.print("Enter name: ");
        String name = sc.nextLine();

        System.out.print("Enter email: ");
        String email = sc.nextLine();

        System.out.print("Enter password: ");
    }
}

```

Figure 12 MainView.java1

[Type here]

[Type here]

[Type here]


```

public class MainView {
    public void addUser() {
        String password = sc.nextLine();

        boolean result = controller.addUser(name, email, password);
        System.out.println(result ? "User added successfully!" : "Error adding user!");
    }

    public void viewAllUsers() {
        List<User> users = controller.getAllUsers();
        if (users.isEmpty()) {
            System.out.println("No users found.");
        } else {
            for (User u : users) {
                System.out.println(u.getId() + " | " + u.getName() + " | " + u.getEmail());
            }
        }
    }

    public void viewUserById() {
        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        User user = controller.getUserById(id);

        if (user != null) {
            System.out.println("ID: " + user.getId());
            System.out.println("Name: " + user.getName());
            System.out.println("Email: " + user.getEmail());
        } else {
            System.out.println("User not found!");
        }
    }
}

```

Figure 13 MainView.java 2

```

public class MainView {
    public void viewUserById() {
    }

    public void startProgram() {
        while (true) {
            showMenu();
            int choice = sc.nextInt();

            switch (choice) {
                case 1 -> addUser();
                case 2 -> viewAllUsers();
                case 3 -> viewUserById();
                case 4 -> {
                    System.out.println("Exiting...");
                    System.exit(status: 0);
                }
                default -> System.out.println("Invalid choice, try again.");
            }
        }
    }

    public static void main(String[] args) {
        new MainView().startProgram();
    }
}

```

Figure 14 MainView.java3

[Type here]

[Type here]

[Type here]

Step 12: Running Program

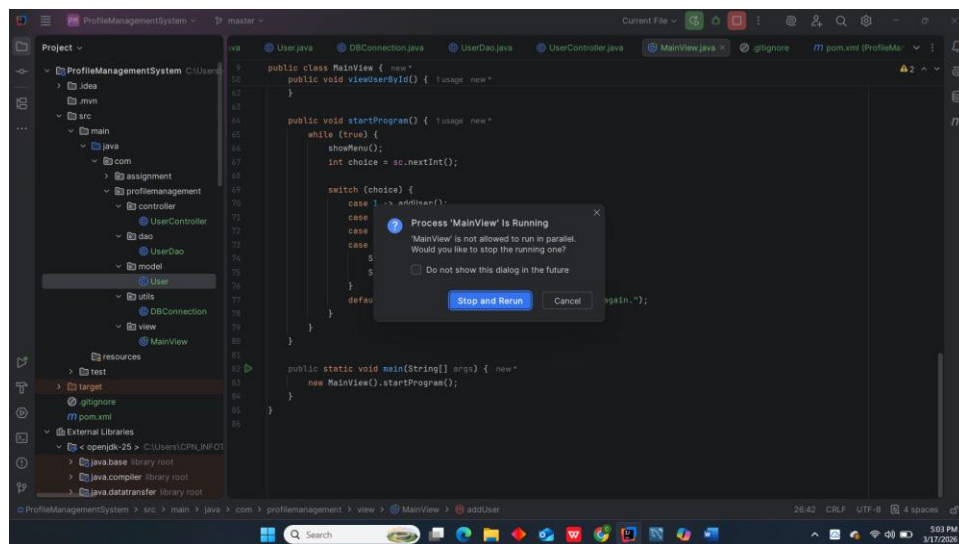


Figure 15 Running program

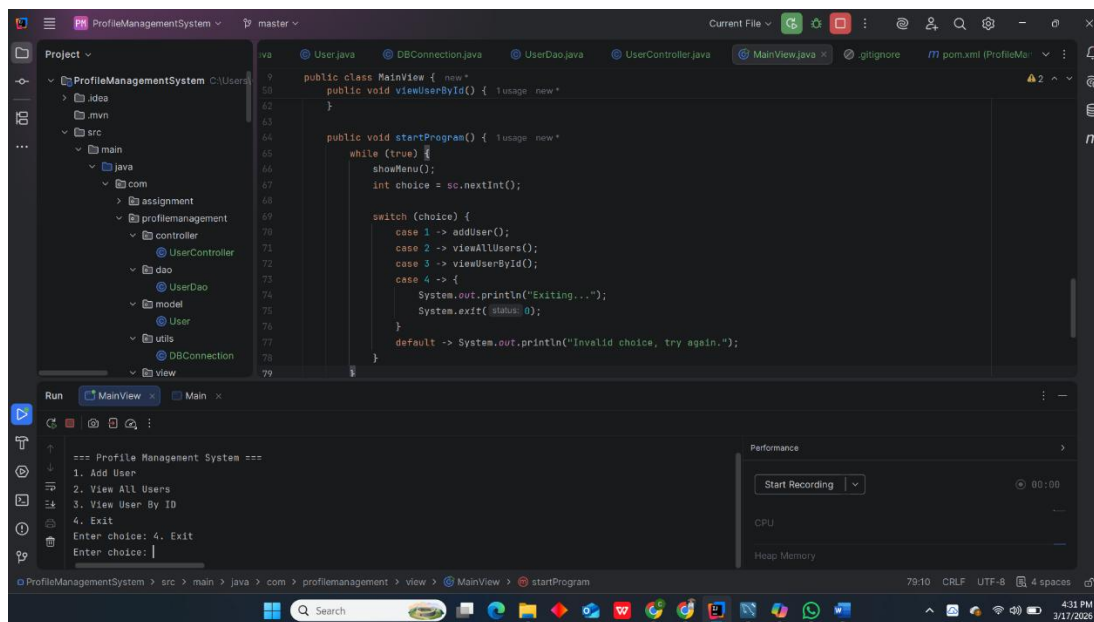


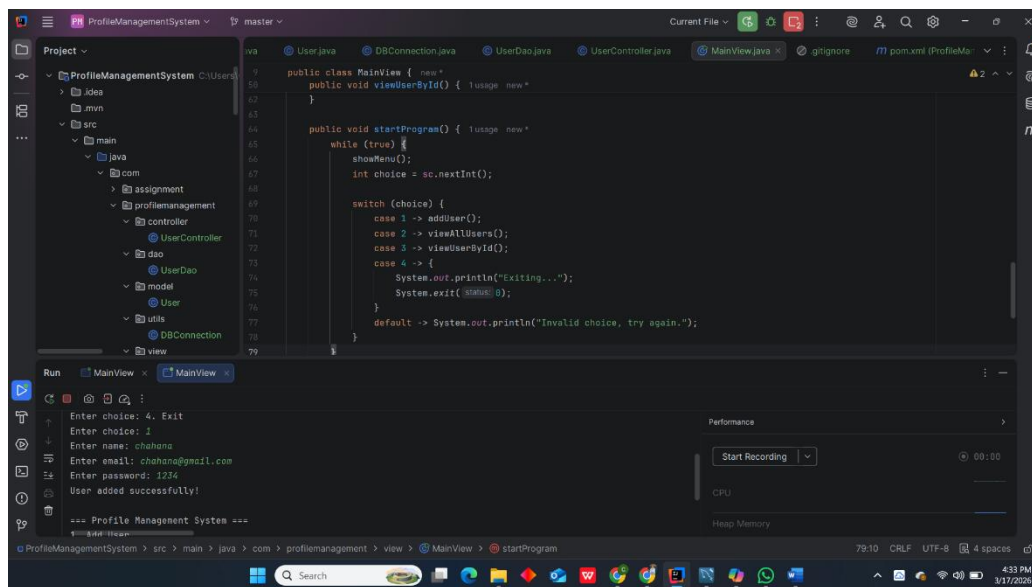
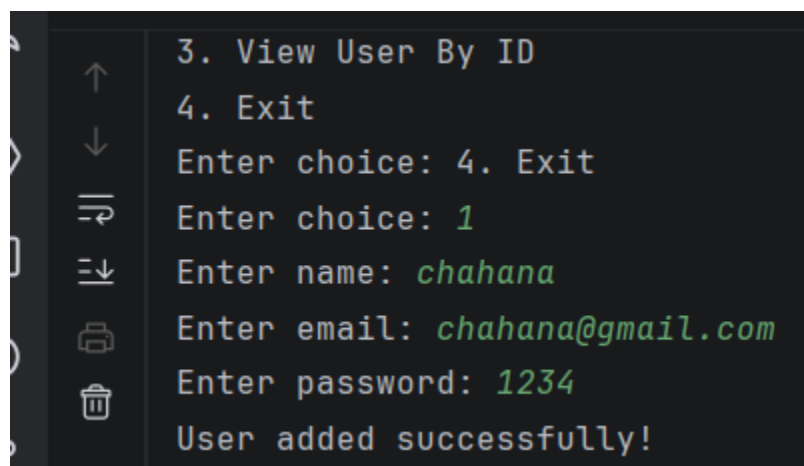
Figure 16 Running Program 2

[Type here]

[Type here]

[Type here]

Step 13: Inserting data

*Figure 17 Inserting data with success message**Figure 18 insertion success in clear view*

Step 14: Fetching all users output

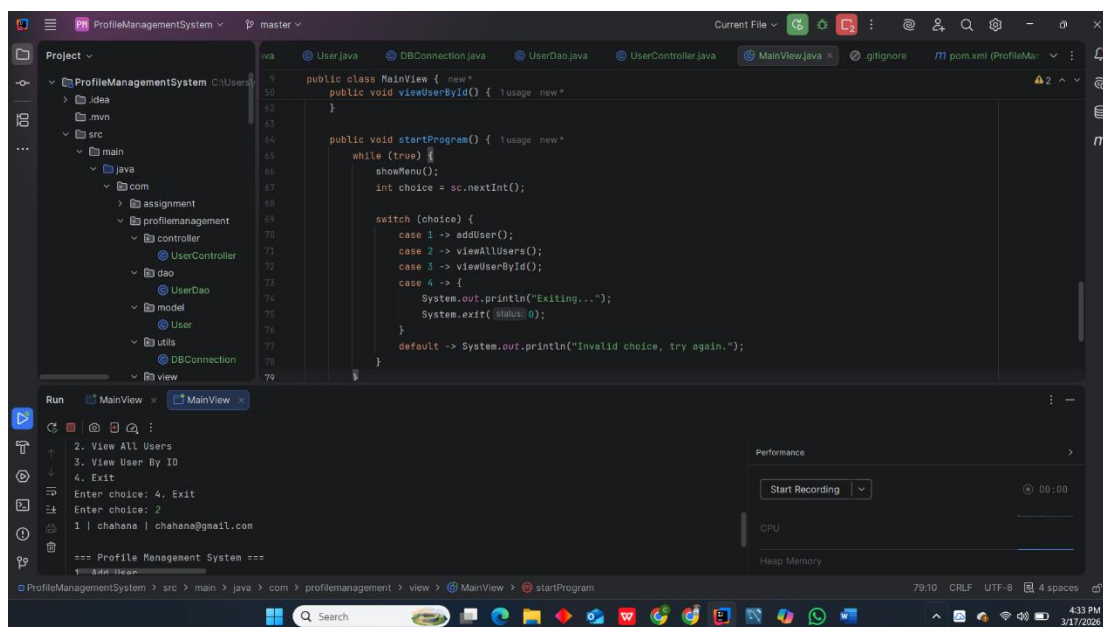


Figure 19 all users output

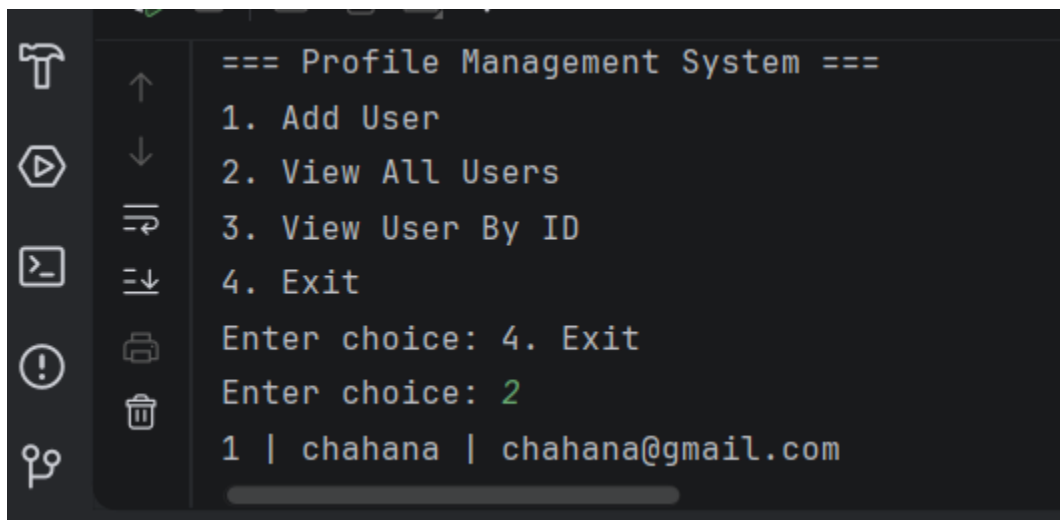


Figure 20 all users in clear view

Step 15: Fetching user by id

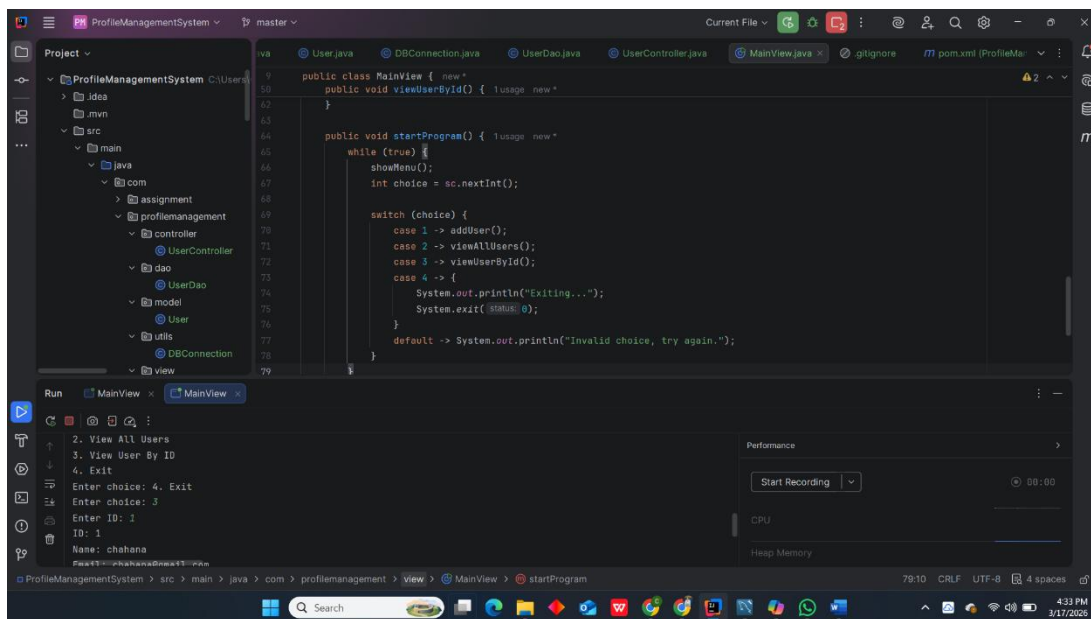


Figure 21 user by id

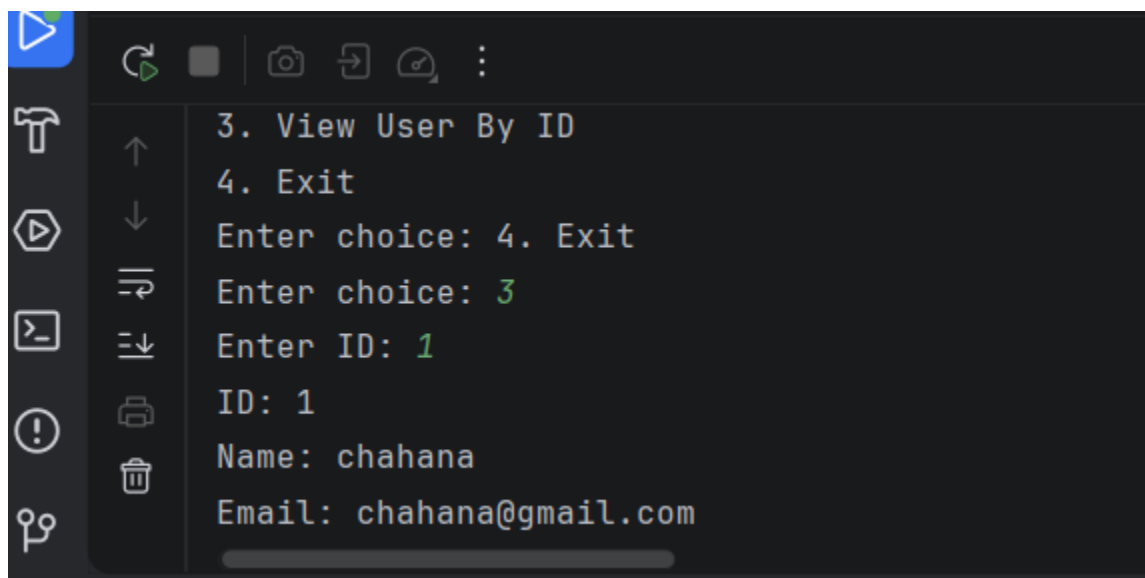


Figure 22 User by id in clear view

Step 16: Existing

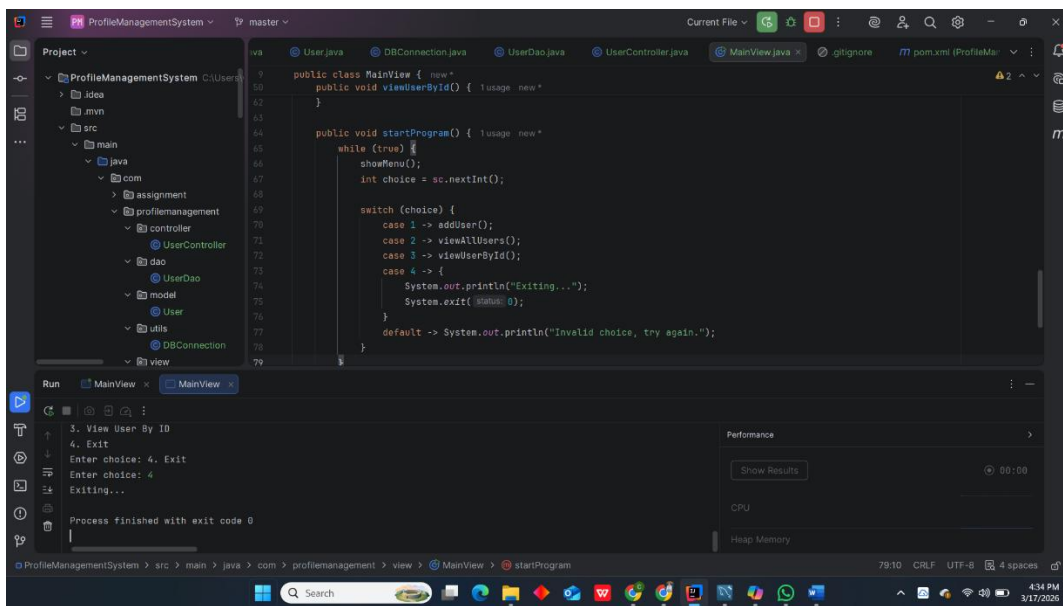


Figure 23 Existing

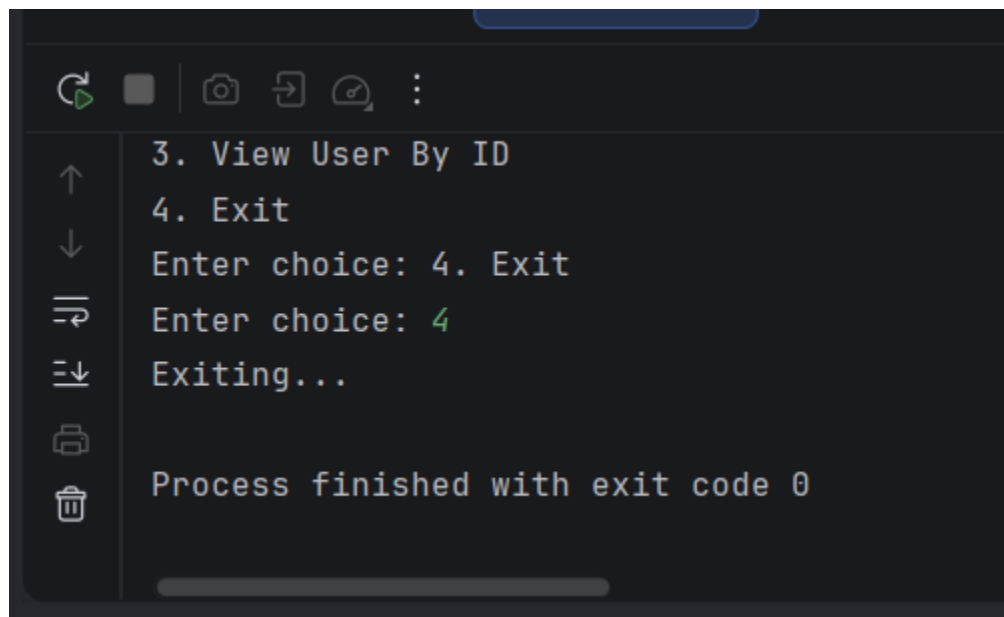


Figure 24 Existing in clear view

4 Conclusion

Working with Java, IntelliJ, MySQL Workbench, MySQL Connector provide me a complete knowledge and experience of building a Profile Management System. I got a practical knowledge of working with these. IntelliJ IDEA is a powerful IDE that streamlined debugging, coding, and project management through maven dependencies and MySQL is a reliable backend database that store, retrieve user information efficiently.